

- a11g-final-submission
 - 1. Video Presentation
 - 2. Project Summary
 - 3. Hardware & Software Requirements
 - Hardware Requirements Specification (HRS)
 - Software Requirements Specification (SRS)
 - 4. Project Photos & Screenshots
 - 5. Codebase

a11g-final-submission

Team Number: 06

Team Name: 404 Firmware Not Found

Team Member Name	Email Address	GitHub Username
Praise Ndlovu	praisen@seas.upenn.edu	pbn107
Anjali Jathavedam	apjath@seas.upenn.edu	apjath

1. Video Presentation

Link: <https://youtu.be/fIFFB9PDKGI>

2. Project Summary

Description:

2 sentence summary:

The DSPiano is a compact, multi-functional piano that allows users to play notes in a four octave range, display the waveform of each note, and record their tune if desired. Users are able to play in free-play mode, or learn how to play sheet music in note-play mode.

Inspiration:

As avid music-enjoyers, we were inspired to build a product that is easy to use and allows for experimentation with music. We have both wanted to learn the piano and thought creating a device that facilitates the learning process would be a great opportunity.

We additionally appreciate that the device is simple enough to use for a very wide range of ages, yet has functionality that will keep it interesting despite age of the user.

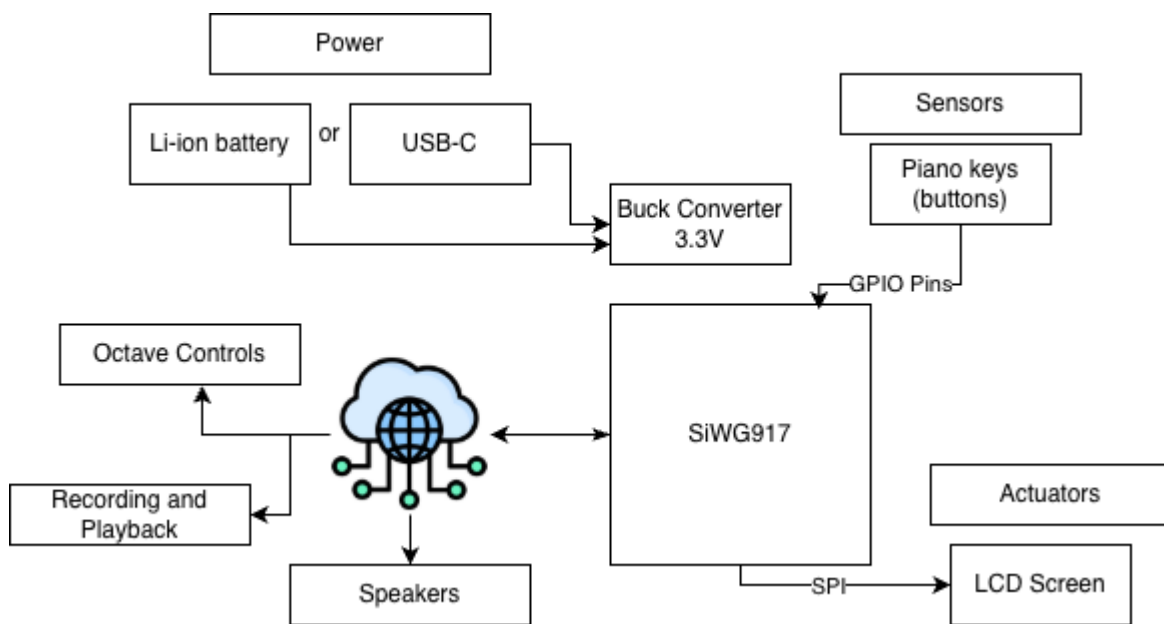
Internet augmentation:

We used Internet connection to allow users to play music through their computer for added portability of the product. Using the Internet also allowed us to create a robust user interface that can be easily navigated and add recording functionality.

Device Functionality and Design:

The user-interface of our device includes the Node-RED dashboard, an LCD screen mounted to the product, and the 8 keys that function as one octave of a piano. Under this layer of abstraction, we created a custom PCBA that can be powered via LiPo or USB connection with power nets for 3.3V and 5V. The PCB has I2C drivers for the buttons and the LEDs, LCD, two microphones, an IMU, an I2S driver, as well as two debuggers, USB data connections, and SD card integration. Our final design did not include the LEDs, microphones, or IMU.

We had one button per note which would fire an interrupt service routine when pressed and send the number of the button via MQTT to Node-RED. The octave was applied in Node-RED and the pure frequency was played through the computer speakers. We created additional functionality in Node-RED to allow the user to record and play back their tunes. In the product itself, the user was able to navigate through different modes and screens on the LCD via the first few buttons.



Challenges:

We faced a few significant challenges throughout the semester. After submitting our PCB files for manufacturing, we were made aware that our I2C footprint could not be matched to any existing I2C expander. We did not consider this possibility during the creation of the board, so did not budget any pins on the chip for general back-up GPIO use. We initially solved this by buying off-board I2C expanders.

Another challenge we faced was the small error of grounding SWO in our schematic, making it impossible to flash to our PCBA. We examined our schematic to find the issue and were able to cut traces on the PCBAs in order to flash to them.

The most significant challenges we faced were with the codebase. We were able to run I2C and SPI separately, but when we attempted to run them together in the integrated codebase, one would fail. We iterated on this problem for over a week, using several different programs for I2C and SPI, but unfortunately we simply could not get both to work at once. We decided to prioritize SPI for our LCD screen and decided to use a development board for GPIO pins to connect our buttons.

The last challenge we faced was with over the air firmware updates (OTAFU). Despite being able to remotely flash to our board in the A08 assignment, we faced difficulty hosting both mqttIn and mqttOut in one WiFi instance in our final prototype. When we kept each process separate, we were able to use our intended Internet functionality and OTAFU. We believe this is because of the difference in protocols with HTTP and HTTPS required by mqttIn and mqttOut, respectively.

Prototype Learnings:

Our initial idea was ambitious and unrealistic for the timeline. We had to adapt and lessened the scope of our project will still pushing ourselves to work hard and learn new skills. We also learned the importance of baking in fall back plans and other options. While creating our schematic and doing PCB layout, we added many layers of functionality like an IMU and two microphones that we did not end up using in the end. In the future we could do a better job of defining the project scope early and being realistic about what could be accomplished in the timeline. We could then allot GPIO pins on the chip to be used as back-ups.

Next Steps & Takeaways:

We were unable to perfect the second mode of play, note-play mode. This mode displayed sheet music and allowed the user to find the corresponding notes to the sheet music in a game style of play. We were able to display notes on the LCD and have them disappear when a key was pressed, but not when the correct key was pressed.

Additionally, we had the original intention of allowing users to sing into a microphone, do DSP on their recording to identify the notes, and then teach the user how to play their song on the keyboard. This would be a significant extension but would fulfill our original goals.

We learned about the entire prototyping process in ESE5160, from ideation to demonstration. We The primary skill we learned was designing PCBs in Altium. It was great to take complete ownership over our PCB and design and discuss design nuances with the manufacturer. We learned about testing the board before usage and the importance of looking back to our schematic and PCB layout in Altium throughout the process. The course was also a great lesson in reading datasheets thoroughly and following manufacturer instructions. Being the first cohort of students to use the Silicon Labs board was a challenge but a certainly rewarding process.

Project Links:

Node-RED instance: <http://52.159.114.54:1880/>

Final PCBA on Altium 365: <https://upenn-eselabs.365.altium.com/designs/51844DEE-0BD5-40C2-A679-D60E8B9CA19E>

3. Hardware & Software Requirements

Hardware Requirements Specification (HRS)

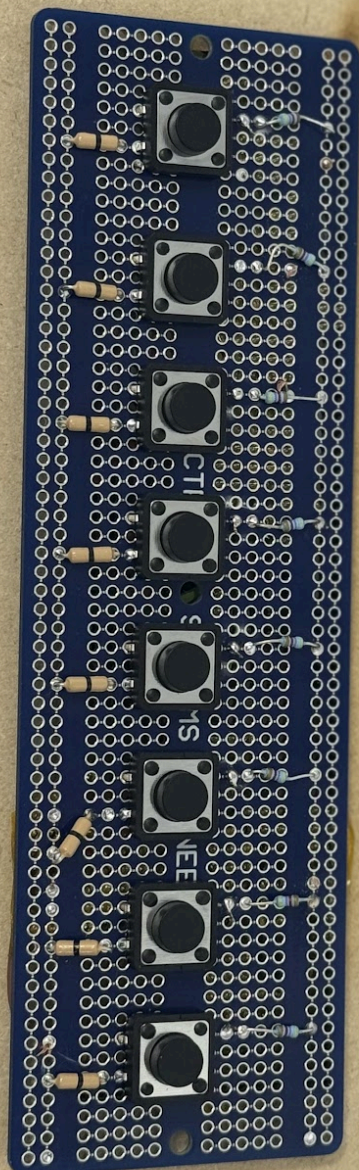
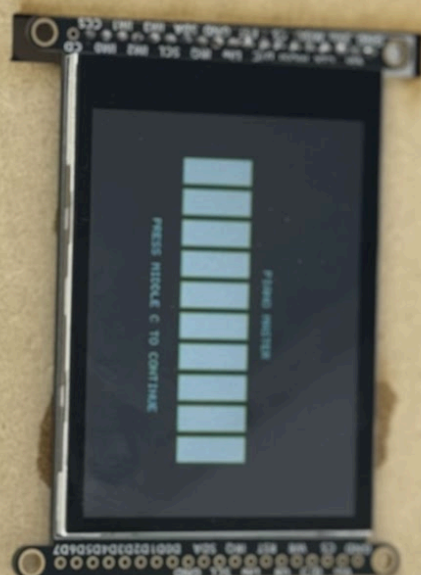
ID	Description	Achieved?	Notes
HRS-01	The device shall play the correct note on the speaker when a key is pressed.	Partially	The original intention was to play notes through physical speakers via I2S. We transitioned to using computer audio through Node-RED.
HRS-02	The LCD screen shall display the notes that the user has already played in Standard mode.	Partially	The LCD screen displayed the current note the user was playing and did not have memory of the previous notes played in free-play mode.
HRS-03	The buttons under the keys shall be used to determine when a key is pressed and can be used for how long it is pressed.	Partially	We successfully used the buttons to determine the note played but we made each note a quarter note (of the same duration).)
HRS-04	The microphone should be used to take in the sound signal from the user.	No	The microphones were a part of our second mode of operation which we did not implement.
HRS-05	A switch should be used to toggle between Standard and Reverse mode.	No	We implemented one mode of operation so did not require a switch to be used.
HRS-06	An LED driver should be used to supply power to the correct key when it is supposed to be pressed, and stop supplying current to this LED after it is pressed.	No	The LEDs were part of the second mode of operation which we did not implement.

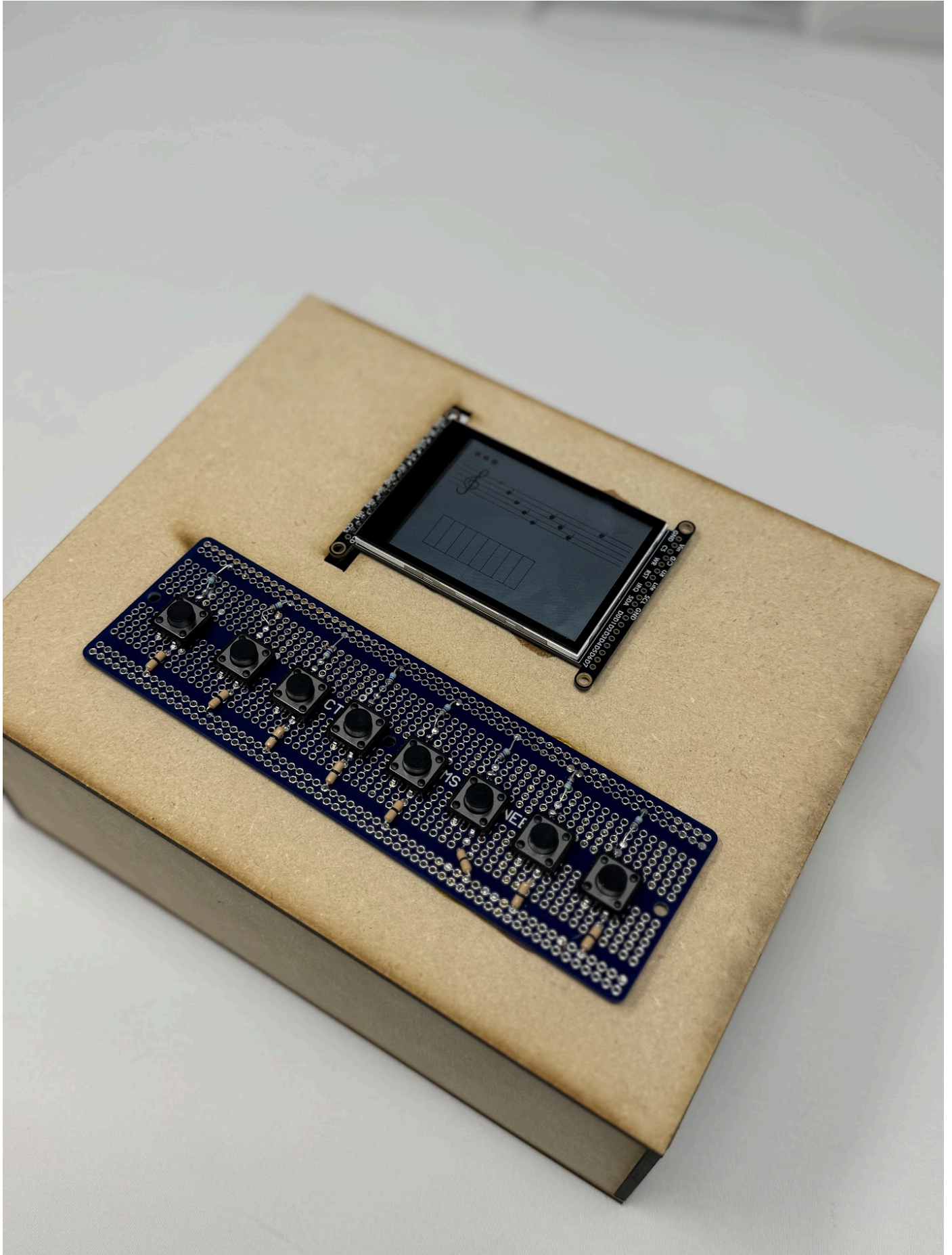
Software Requirements Specification (SRS)

ID	Description	Achieved?	Notes
SRS-01	[secondary] The program may transmit the recorded audio to the cloud to extract the notes. The notes will be transmitted to the MCU.	No	Note extraction was part of our second mode of operation which we did not implement.
SRS-02	The notes that the user plays on the keyboard shall be transmitted to the online system in succession.	Yes	Each note was successfully transmitted to the online system via MQTT.
SRS-2.5	The SD card shall store songs that can be played on the speaker of the module.	No	We did not use the SD card.
SRS-03	The duration that each note is played for should be transmitted to the online system.	No	We did not record duration of the notes played.
SRS-04	The software should light the correct LED when it should be lit, and turn it off after it is pressed. The software shall communicate with the LED Driver via I2C.	No	The LEDs were part of the second mode of operation which we did not implement.
SRS-05	The device shall use an LCD to display the notes in our original UI.	Yes	We created an original UI for our LCD with a music staff and keyboard representation.

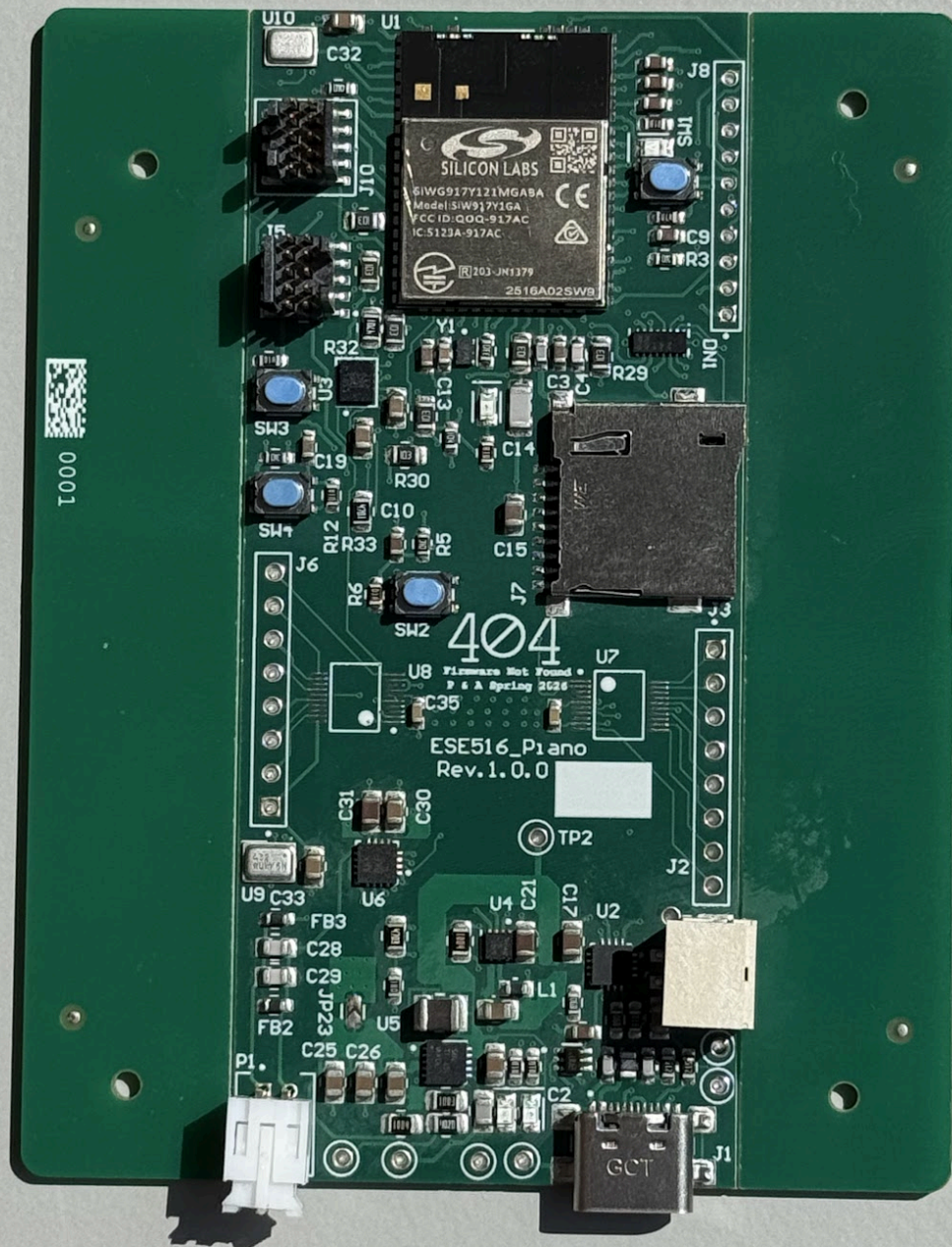
4. Project Photos & Screenshots

Final prototype:

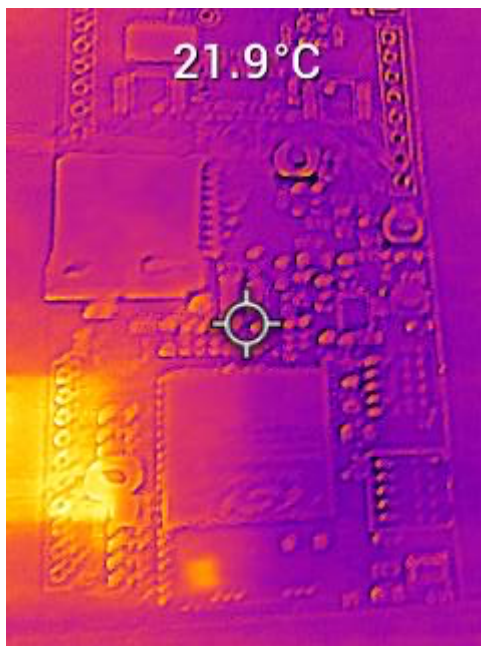
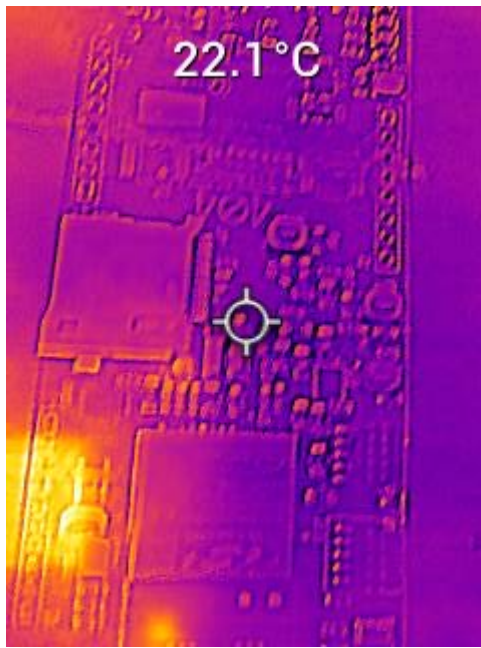




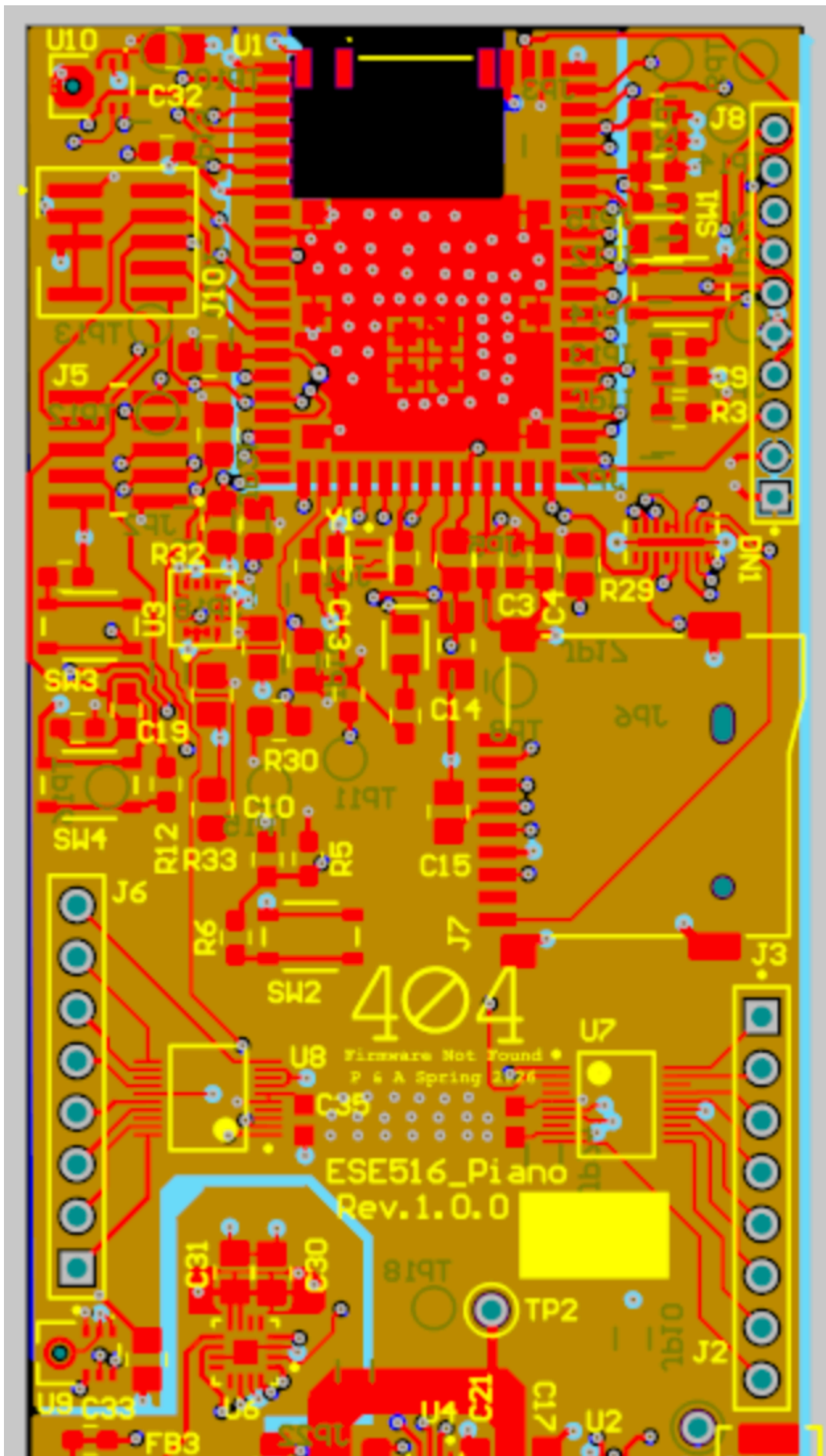
PCBA, top:

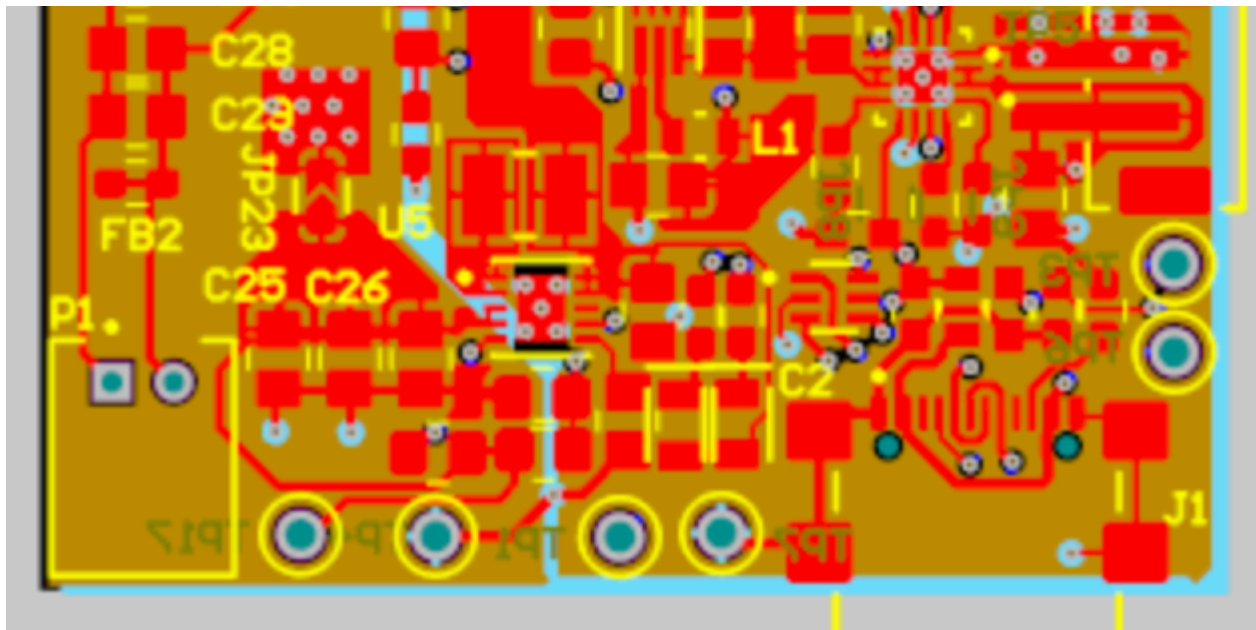


PCBA, bottom:

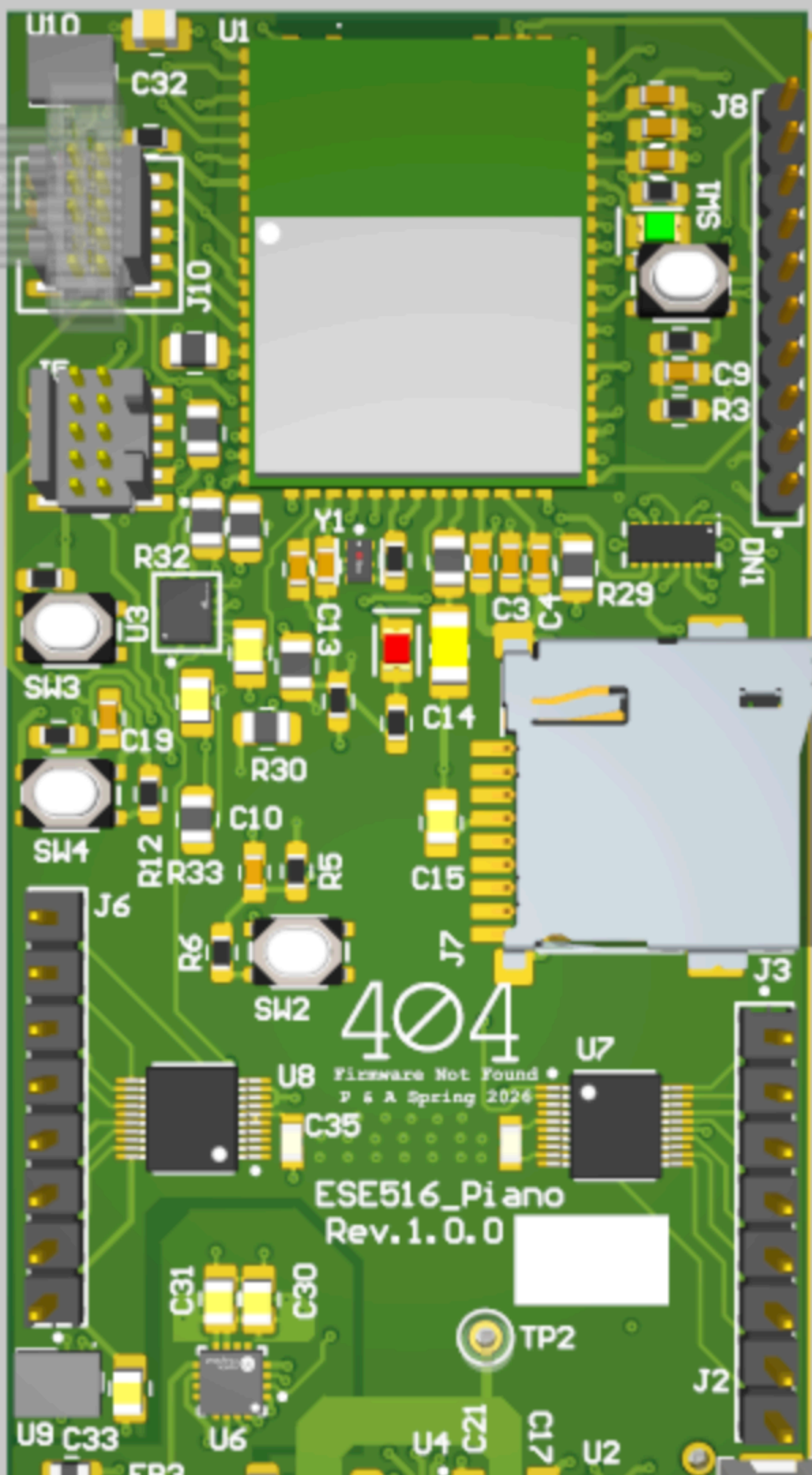


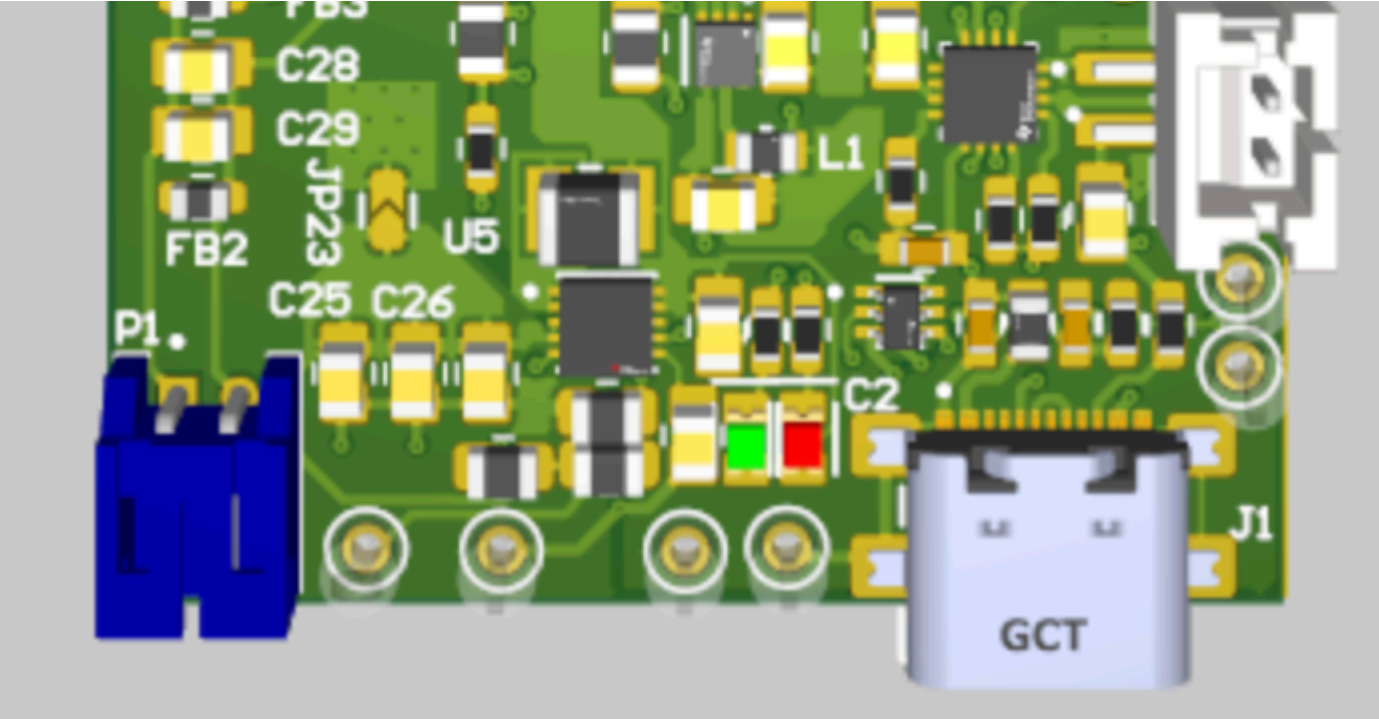
2D PCBA Layout:



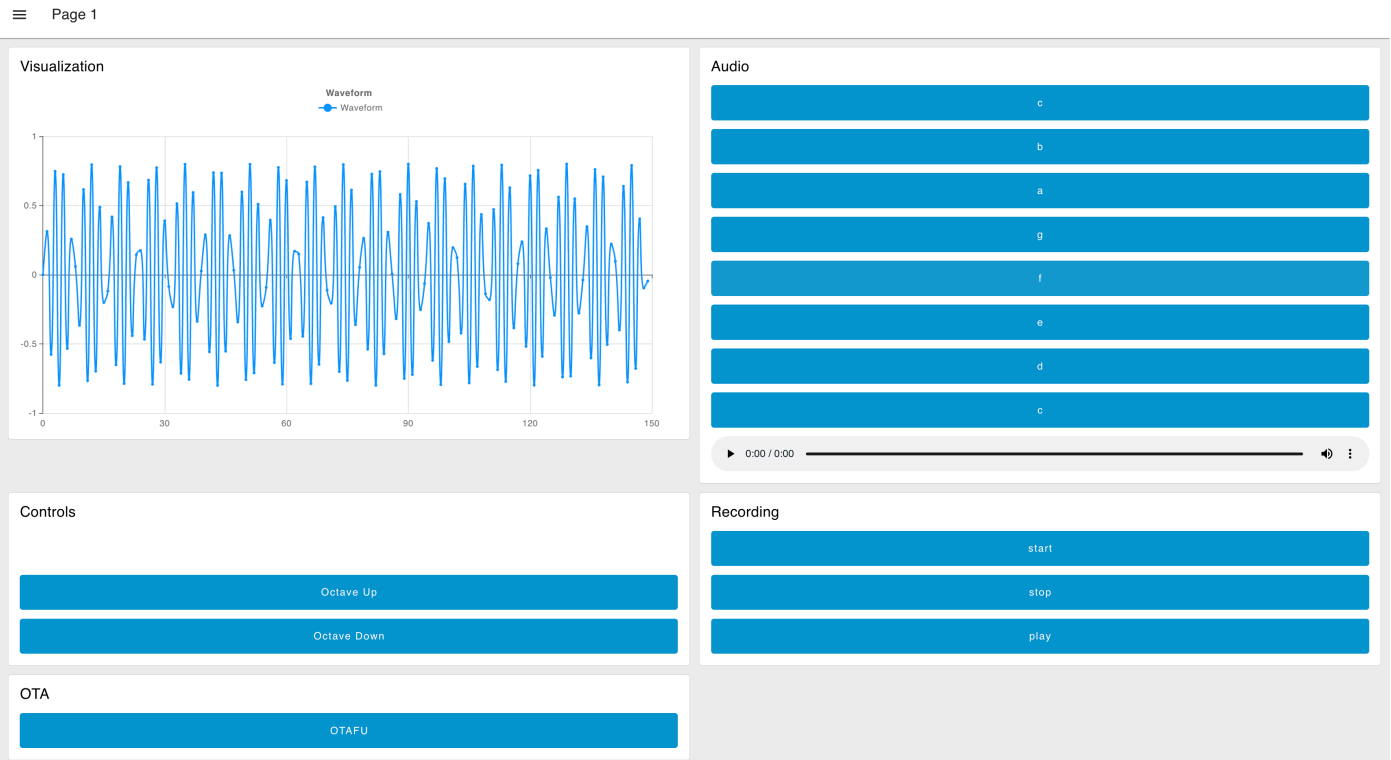


3D PCBA Layout:





Node-RED Dashboard:



Node-RED Backend:

